

The posix Portfolio, *Measured*

Generated 2026-09-04 · 15 sections · every figure traces to a CSV/JSON in this run's output directory

posx — Repository Analysis

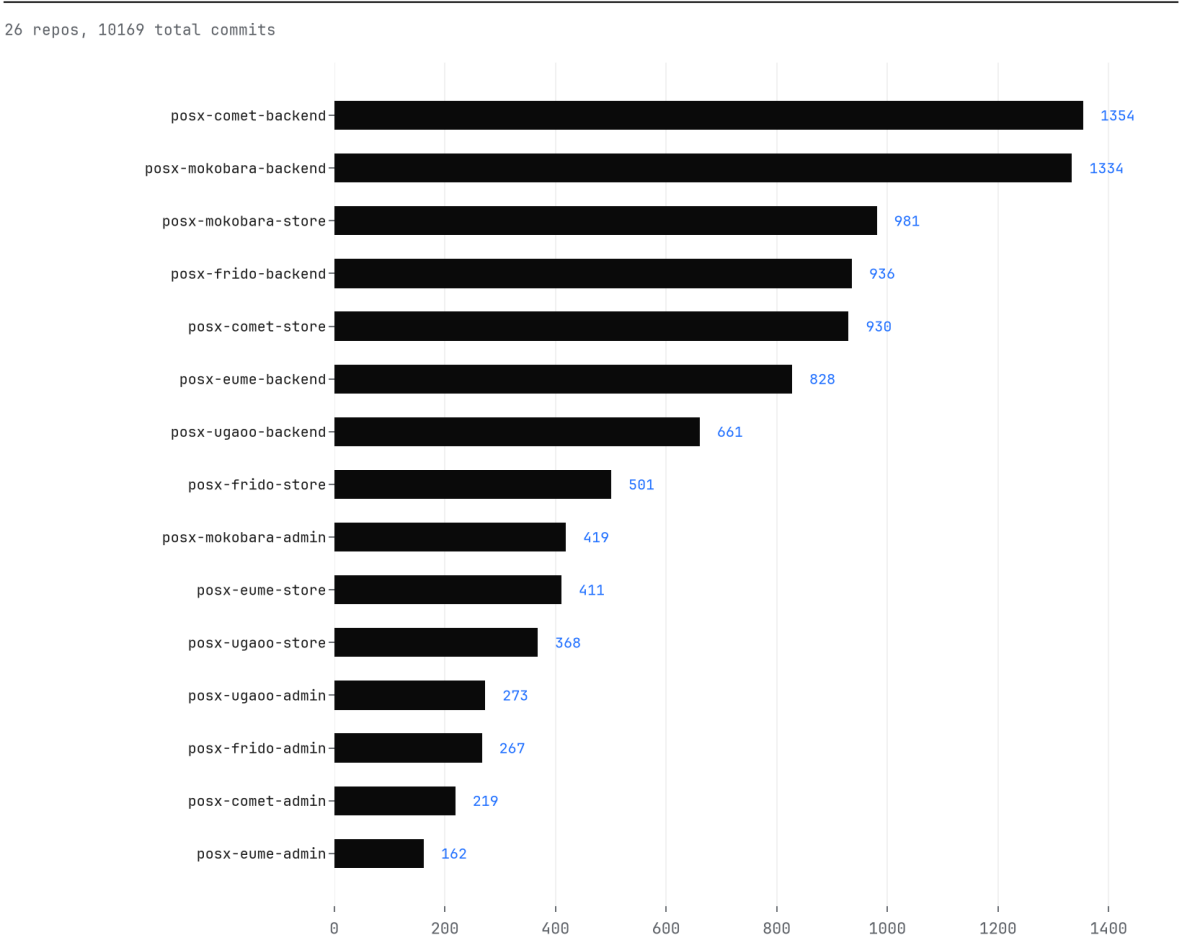
Generated from `git log` history in each of 26 repos. Tiers are by days since last commit; bus-factor is the Gini coefficient of each repo's per-author commit share (0 = evenly spread, 1 = one author owns everything). Data: `inventory.csv` . Chart: `charts/repo_activity_top15.png` .

1. Distribution at a glance

BUCKET	DEFINITION	COUNT
Active	<=90 days since last commit	26
Recent	90d-1yr	0
Aging	1-3yr	0
Dormant	>3yr	0

Total commits across the portfolio: **10169**. Median bus-factor Gini across all repos: **0.596**.

posx: commit volume by repo



2. Most active repos by volume

REPO	COMMITTS	AUTHORS	BUS-FACTOR GINI	TOP AUTHOR SHARE
posx-comet-backend	1354	27	0.6633	0.2821
posx-mokobara-backend	1334	26	0.7902	0.3913
posx-mokobara-store	981	24	0.7212	0.3242
posx-frido-backend	936	21	0.6968	0.3141
posx-comet-store	930	26	0.6772	0.2774
posx-eume-backend	828	23	0.6468	0.2995
posx-ugao0-backend	661	16	0.7108	0.3646
posx-frido-store	501	19	0.5826	0.2335
posx-mokobara-admin	419	23	0.6998	0.3866
posx-eume-store	411	21	0.4729	0.1995
posx-ugao0-store	368	14	0.6848	0.3967
posx-ugao0-admin	273	13	0.6374	0.4432
posx-frido-admin	267	16	0.6084	0.3483
posx-comet-admin	219	20	0.54	0.3607
posx-eume-admin	162	15	0.484	0.1975

3. Bus-factor / key-person risk

Repos where one author holds more than 70% of all commits -- the highest key-person risk in the portfolio:

REPO	TOP AUTHOR SHARE	AUTHORS	COMMITTS
posx-kb	1.0	1	5
posx-mokobara-clearance-backend	0.94	2	50
posx-frido-b2b-store	0.8906	3	64
posx-frido-b2b-backend	0.7833	6	120
posx-frido-b2b-admin	0.7619	2	42

4. Honest limitations

- Tiering uses raw last-commit date; this tool does not attempt to detect and exclude org-wide automated batch commits (e.g. a mass tooling migration) the way a human analyst reviewing the corpus might. A single such batch commit across many repos would understate how dormant those repos really are -- check `last_commit` against `top_author` for repos where they look suspicious before trusting the tier.
- Bus-factor Gini is computed over the repo's *entire* history, not a recent window -- a repo that was single-author at launch and later grew a real team will still show an elevated historical Gini.

Raw data: `inventory.csv`.

posx — *Architecture*, HLD & Memory Graph

Built via `codebase-memory-mcp` (LSP-based call/usage resolution -- a real persistent code knowledge graph, i.e. a "memory graph" of the codebase, not a static-analysis approximation of one). This capture: 8199 nodes, 14599 edges across 3 repo(s): `posx-mokobara-backend`, `posx-mokobara-admin`, `posx-mokobara-store`. Caveat, said plainly: this capture used an interactive MCP tool available in a Claude Code session, not a subprocess `chronicle-analyzer`'s CLI can invoke standalone on any machine -- treat this section as a snapshot from whatever repos were indexed when it was captured, not a rerunnable module output the way every other section in this report is. Re-capture it for a different repo set by indexing with `codebase-memory-mcp` and re-saving `architecture_graph.json` / `shared_deps.json` in this format -- see `docs/METHODOLOGY.md` .

1. Inter-repo connectivity: measured, not assumed

The tool's own cross-repo-intelligence mode matches HTTP/async/gRPC/GraphQL/tRPC calls across indexed projects. Result across 11 projects: **0 cross-repo calls found** (HTTP: 0, async: 0, channel: 0, gRPC: 0, GraphQL: 0).

Most-shared internal signal: `moment` is used by **25 of 26 repos** -- check whether it is a real shared internal package (worth checking its registry listing/publish count) rather than assuming no cross-repo dependency exists.

PACKAGE	REPOS USING IT
moment	25
axios	23
zod	18
react	17
react-dom	17
@devxcommerce/shopify	17
cmdk	16
@hookform/resolvers	16
date-fns	16
react-hook-form	16
@tanstack/react-query	9
nodemailer	9

2. HLD: layers and boundaries (posx-mokobara-backend)

PACKAGE	LAYER	WHY
api	entry	has entry points, only outbound calls
jobs	internal	fan-in=3, fan-out=25
json	api	has HTTP route definitions
lib	core	high fan-in (879 in, 7 out)
modules	core	high fan-in (113 in, 37 out)
subscribers	entry	only outbound calls
workflows	entry	only outbound calls

FROM	TO	CALL COUNT
api	lib	809
api	modules	72
modules	lib	37
subscribers	lib	23
subscribers	modules	16
jobs	modules	15
jobs	lib	10
lib	modules	7
workflows	modules	3
api	jobs	3

api → lib : 809 calls -- the largest internal boundary found in this capture; cross-check against `DUPLICATION.md` for whether that target package is also duplicated across repos.

3. LLD: the actual hotspot functions and real clusters

FUNCTION	FAN-IN (CALLERS)
api-response.sendApiResponse	249
api-response.createSuccessResponse	243
api-response.createErrorResponse	240
calculate-online-coupon-discount.warn	46
calculate-store-coupon-discount-step.warn	45
calculate-online-coupon-discount.log	43
build-summary.buildSummary	23
UnicommerceService.init	23
get-s3-file-path-for-order.getS3FilePath	18
date-utils.startOfDayIST	14

sendApiResponse (249 callers) is the single most depended-upon function found in this capture.

Real Leiden-detected clusters (the de-facto modules, which cut across folder layout):

CLUSTER	MEMBERS	COHESION	REPRESENTS
10	204	0.76	sendApiResponse, createSuccessResponse, createErrorResponse
6	107	0.75	log, init, init
0	58	0.5	warn, PUT, POST
50	46	0.69	redeem, writeAudit, cancelVoucher
3	44	0.89	call, redeem, cancelRedeem
7	42	0.69	startOfDayIST, buildInwardsDigest, endOfDayIST
2	34	0.87	POST, evaluateShopifyOffer, evaluateRetailOffer

CLUSTER	MEMBERS	COHESION	REPRESENTS
64	24	0.92	hasPermissionByRole, hasPermission, resolveActorEmployee

4. Honest limitations

- Representative, not exhaustive: the indexed repo(s) stand in for any structurally similar repos elsewhere in the portfolio (check `DUPLICATION.md` for which repos actually share structure before generalizing this section to them).
- Cross-repo-intelligence only detects *code-level* calls (HTTP/async/RPC). It cannot detect connectivity through shared infrastructure with no code reference -- this analysis has no runtime-wiring export (e.g. a CTO-approved production env-var dump) for this target, so infra-level connectivity (if any) is unverified, not ruled out.

Raw data: `architecture_graph.json` , `shared_deps.json` .

posx — Knowledge Graph

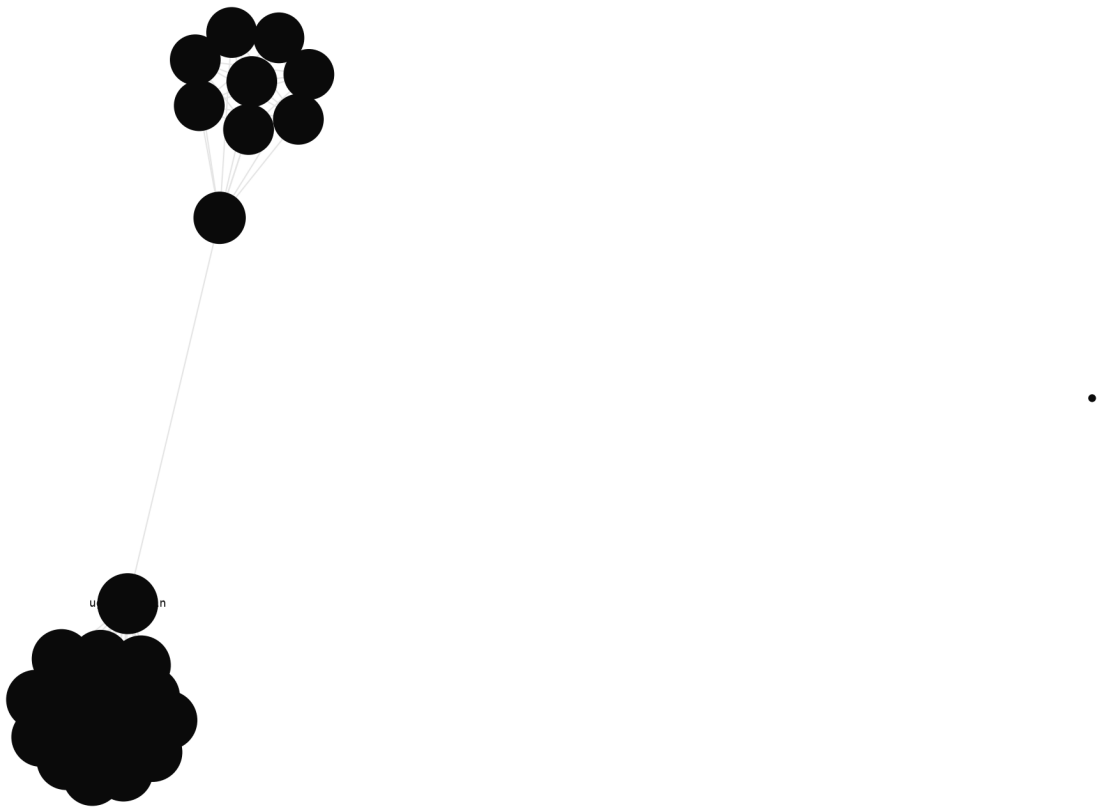
A real, exported graph artifact -- `knowledge_graph.graphml` -- not a description of one. Standard GraphML: loads directly in Gephi, yEd, Neo4j (`neo4j-admin database import`), igraph, or networkx (`nx.read_graphml(...)`). Built by assembling every relationship this tool has already computed into one graph, re-scanning nothing: *DUPLICATE_OF* edges (repo<->repo, weighted by count of byte-identical files -- `exact_duplicates.py` 's sha256 output, not a heuristic), *SHARES_PACKAGE** edges (repo<->repo, weighted by shared npm dependency count, =5 floor), *COUPLED_WITH* edges (file<->file within one repo, weighted by code-maat's temporal-coupling degree). Round-trip verified: exported, reloaded, and spot-checked against known-real duplication counts before being reported here.*

1. What's in it

1075 nodes (26 repos, 1049 files that participate in at least one coupling relationship), **2195 edges**:

EDGE TYPE	COUNT
DUPLICATE_OF	186
SHARES_PACKAGE	312
COUPLED_WITH	1697

posx: repo-to-repo knowledge graph (duplication + shared packages)



2. Most-connected repos

REPO	DEGREE (DUP + SHARED-PACKAGE CONNECTIONS)
posx-ugao-admin	46
posx-mokobara-store	44
posx-comet-store	44
posx-frido-store	44
posx-mokobara-admin	44
posx-eume-store	44
posx-ugao-store	44
posx-frido-admin	44
posx-comet-admin	44
posx-eume-admin	44

3. How to query it yourself

```
import networkx as nx
G = nx.read_graphml("knowledge_graph.graphml")
# every repo this one shares 100+ identical files with:
[(u, v, d["weight"]) for u, v, d in G.edges("repo:posx-ugao0-admin", data=True)
 if d.get("type") == "DUPLICATE_OF" and d["weight"] > 100]
```

4. Honest limitations

- File-level nodes exist only for `COUPLED_WITH` (within-repo) relationships -- the portfolio's full within-repo import graphs (thousands of file nodes each) live separately in `depgraph_raw/*.json` per repo, not merged into this graph, to keep it at a size actually useful to open in a graph tool rather than an unreadable hairball.
- `SHARES_PACKAGE` is direct dependencies only (not transitive) and floors at 5+ shared packages to filter noise (every repo shares *some* common package).
- No cross-repo `CALLS` edges are in this graph -- `ARCHITECTURE.md`'s codebase-memory-mcp capture already established that number is zero for the 3 repos it indexed.

Artifact: `knowledge_graph.graphml` . *Raw stats:* `knowledge_graph_stats.json` .

posx — Commit *Ontology*

Every non-merge commit classified by a deterministic rule table (file-pattern rules first, then conventional-commit prefix / keyword match, `other` when neither matches -- see `docs/METHODOLOGY.md` for the exact rules and priority order). No LLM in the loop, fully reproducible. Data: `ontology_commits.csv` , `ontology_summary.json` .

1. The headline: where the work actually goes

7137 non-merge commits classified, **17.58% left honestly unclassified** rather than force-fit into a category.

SUPERCLASS	% OF COMMITS
delivery	34.87%
correction	31.81%
other	17.58%
code_health	7.66%
ops_config	5.86%
housekeeping	1.44%
data_schema	0.77%

Delivery (34.87%) vs. Correction (31.81%): correction is nearly as large as delivery -- for every commit that ships something new, a comparable amount of work is spent fixing something. Ops & config sits at 5.86% of all commits -- low, consistent with a young codebase that hasn't yet accumulated infrastructure sprawl.

2. Full leaf-type breakdown

LEAF TYPE	COUNT
feature	2489
bug_fix	2223
other	1255
refactor	399
ci_build	388
docs	100
chore	92
data_schema	55
revert	47
test	36
access_admin	25
perf	12
style	11
dependency_bump	5

3. Honest limitations

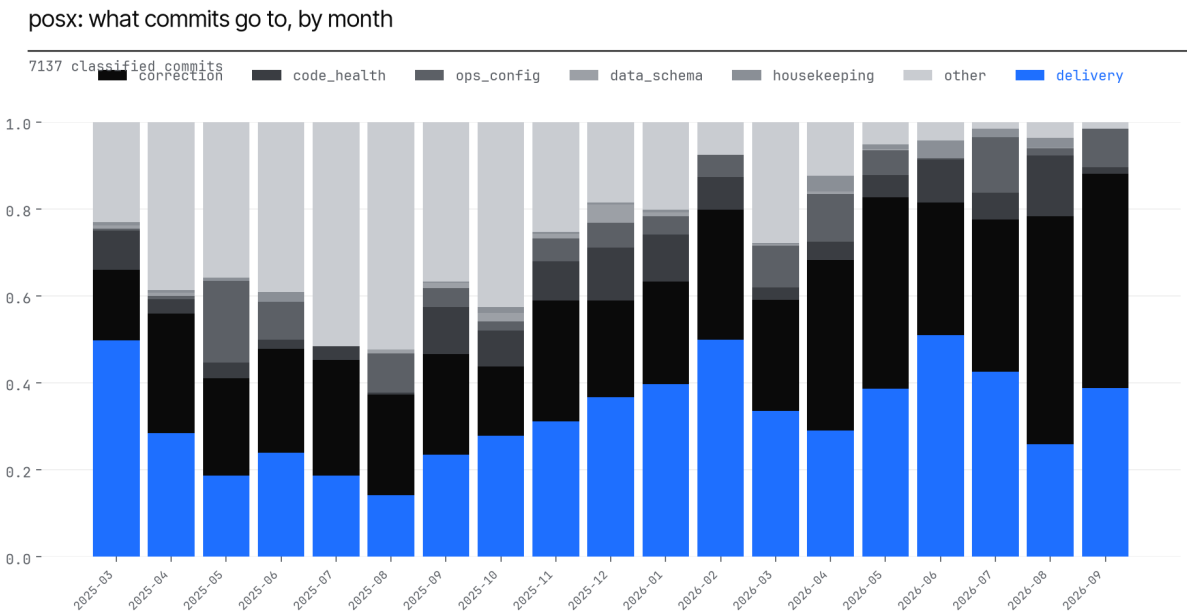
- The classifier's "other" bucket (17.58%) is deliberately conservative: an ambiguous, jargon-heavy commit subject is left unclassified rather than guessed into a category (see `ONTOLOGY_AUDIT.md`'s disagreement analysis for the specific asymmetry this produces against a more liberal human rater).
- File-pattern rules only fire when *all* files a commit touched match one pattern; a mixed commit (e.g. a lockfile bump alongside a real code change) falls through to message-based classification, which may miscategorize it.

Raw data: `ontology_commits.csv` . Run `ontology_audit` (see `README`) to independently verify this classifier's accuracy on your own corpus before trusting the headline percentages.

posx — Engineering Effort *Allocation*

Basis: all 7137 non-merge, ontology-classified commits, joined with authorship and the files each commit touched. Toil clusters are found mechanically (grouped by repo + directory + category, flagged above 15 commits) -- each is a candidate worth a human diff-read, not a verified finding the way a hand-audited deep-dive would be. Data: `effort_by_author.csv` , `effort_monthly_share.csv` , `effort_toil_clusters.csv` . Chart: `charts/effort_share_over_time.png` .

1. Where the commits go, over time



28 authors have 20+ commits; their median Delivery share is 27.19%.

2. Candidate toil clusters

72 clusters found, covering 3060 commits (42.9% of all classified work) -- repeated, mechanical, same-location work that isn't new-feature delivery:

REPO	LOCATION	CATEGORY	COMMITTS	AUTHORS	SPAN
posx-mokobara-store	src/components	correction	136	11	2025-11-13 to 2026-09-03
posx-mokobara-backend	src/workflows	correction	135	8	2025-11-20 to 2026-09-02
posx-comet-store	src/components	correction	121	12	2025-03-16 to 2026-08-31
posx-comet-backend	src/workflows	correction	108	12	2025-03-18 to 2026-08-26
posx-comet-backend	src/workflows	other	85	7	2025-03-13 to 2026-07-02
posx-frido-backend	src/workflows	correction	81	6	2025-07-23 to 2026-09-03
posx-mokobara-store	src/components	other	80	8	2025-10-28 to 2026-08-24
posx-eume-backend	src/workflows	correction	72	8	2025-08-29 to 2026-08-26

REPO	LOCATION	CATEGORY	COMMITTS	AUTHORS	SPAN
posx-frido-backend	src/api	correction	70	4	2025-07-28 to 2026-08-26
posx-frido-store	src/components	other	70	6	2025-07-11 to 2026-04-20
posx-mokobara-backend	src/api	correction	68	9	2025-11-19 to 2026-09-02
posx-mokobara-store	src/pages	correction	67	7	2025-11-28 to 2026-09-03
posx-comet-store	src/components	other	66	7	2025-03-17 to 2026-08-25
posx-frido-backend	src/workflows	other	66	5	2025-07-18 to 2026-06-22
posx-mokobara-admin	src/routes	correction	65	9	2025-11-21 to 2026-09-02

3. Per-author breakdown (top 12 by volume)

AUTHOR	COMMITTS	DELIVERY %	CORRECTION %	OPS/CONFIG %
kaushal.padaliya@devxconsultancy.com	1208	32.12	36.09	2.57
rachit.shah@devxconsultancy.com	1068	51.69	32.58	1.59
yashkumar.parmar@devxlabs.ai	784	49.87	20.92	1.91
dhruv.parekh@devxconsultancy.com	434	27.19	30.65	0.46
vishal.makwana@devxconsultancy.com	404	26.73	22.52	6.19
jaydeep.pipaliya@devxlabs.ai	337	25.82	58.16	8.31
karan181716@gmail.com	302	9.6	12.58	0.33
kritik.jiyaviya@devxconsultancy.com	297	69.02	9.43	0.34
vinay.software.com@gmail.com	255	48.24	39.61	0.0
262339854+jaydeep-devx@users.noreply.github.com	226	42.04	49.12	0.0
jinang.vohera@devxlabs.ai	209	25.84	64.59	2.39
bhagyashree@devxconsultancy.com	195	6.67	20.0	16.92

4. Honest limitations

- Clusters are grouped by directory prefix and category only -- two unrelated recurring patterns in the same directory (e.g. both config tweaks and bug fixes under one feature's folder) will merge into one cluster row. Read `sample_subjects` in the raw CSV before citing a cluster's size as evidence of one specific problem.
- No effort/build-cost estimate is attached to any cluster (unlike a hand-authored toil deep-dive) -- that requires product/ownership context this tool does not have. Use the cluster list to prioritize *which* areas to investigate, not as a ready-made remediation plan.
- Per-author percentages are a commit-count proxy for effort, not a performance measure -- commit size varies enormously by type of work; a single infra commit can be larger than ten feature commits combined. Do not rank authors by this table.

Raw data: `effort_by_author.csv` , `effort_monthly_share.csv` , `effort_toil_clusters.csv` .

posx — Defect *Escape* Rate (SZZ)

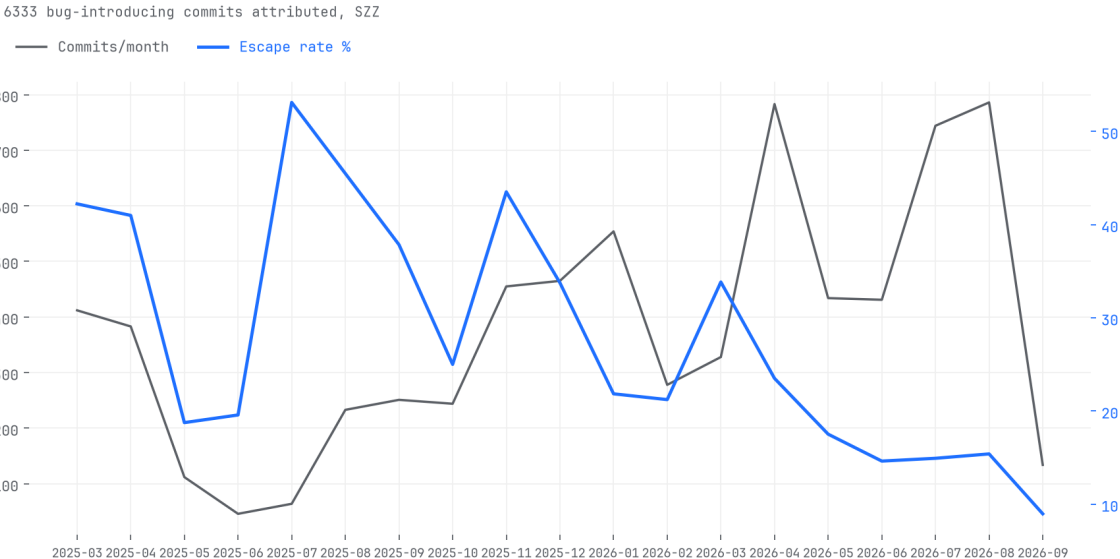
A defect escape is dated to the commit that introduced it, not the commit that fixed it -- that is the moment whatever testing/review/CI existed on that day either caught it or didn't. Implemented via the SZZ algorithm (Sliwerski/Zimmermann/Zeller): for each non-merge commit this tool's ontology classifier calls `bug_fix` or `revert`, `git blame` at the fix's parent commit attributes the deleted/modified lines back to their origin. Window: 365 days. Data: `escapes.csv`, `escape_monthly.csv`, `escape_summary.json`. Chart: `charts/escape_trend.png`.

1. Results

6333 bug-introducing commits attributed. Fix latency: median **22 days**, p90 **242 days** -- one defect in ten survives that long before its fix lands.

The trend is **declining**: the earliest observed cohorts ran 19-42%, the most recent observed months run 9-15%.

posx: commits vs. defect escape rate



2. By repo (highest attributed escape count)

REPO	ATTRIBUTED ESCAPES
posx-mokobara-backend	979
posx-comet-store	917
posx-comet-backend	801
posx-mokobara-store	740
posx-frido-backend	666
posx-eume-backend	361
posx-ugao-backend	353
posx-ugao-store	336

REPO	ATTRIBUTED ESCAPES
posx-mokobara-admin	267
posx-eume-store	235

3. Honest limitations

- This is line-based SZZ without the meta-change/line-mapping refinements from SZZ RA/SZZ Unleashed, which correct for lines that moved rather than changed and filter cosmetic reformatting -- expect some inflation toward reformatting-adjacent commits.
- Detection bias is irreducible in git-only data: an escape is only visible once someone wrote a fix commit for it. A repo with less code review scrutiny will always measure artificially safer than it really is.
- Depends entirely on the ontology classifier's `bug_fix / revert` recall -- if that classifier under-detects fixes (check `ONTOLOGY_AUDIT.md`), escape rates here are a conservative floor, not the true rate.
- Pure-addition fix commits (no deleted lines) carry no blame target and are excluded from attribution -- see `escape_summary.json`'s `per_repo_fix_commit_stats` for the excluded count per repo.

Raw data: `escapes.csv`, `escape_monthly.csv`.

posx — Dependency *Graph* & Hotspots

Internal import graph via `dependency-cruiser` (JS/TS), restricted to modules whose own source is not under `node_modules` -- an early pass in building this tool nearly reported hundreds of false "circular dependency" hits that were entirely inside third-party packages' own internals; see `docs/METHODOLOGY.md`. Complexity via `lizard` (all languages) joined against `code-maat` churn: $\text{hotspot_score} = \text{total_ccn}(\text{file}) \times \text{n_revisions}(\text{file})$ (Tornhill). Coupling via `code-maat`'s temporal-coupling analysis (files that change together, independent of any import between them). Data: `depgraph_summary.csv`, `complexity_hotspots.csv`, `churn_coupling.csv`. Chart: `charts/hotspots_top.png`.

1. Internal circular dependencies

20 of 26 repos analyzed (6 skipped -- see below). **19 of 20 have zero internal circular dependencies.**

REPO	INTERNAL MODULES	CIRCULAR DEPS	ORPHANS
posx-comet-admin	3713	0	50
posx-comet-backend	1278	0	18
posx-demo-admin	3500	0	46
posx-demo-backend	843	0	11
posx-eume-admin	3571	0	48
posx-eume-backend	1288	0	18
posx-frido-admin	3789	0	54
posx-frido-b2b-admin	3594	0	49
posx-frido-b2b-backend	971	0	71
posx-frido-backend	1385	0	30
posx-frido-mobility-admin	3613	0	48
posx-frido-mobility-backend	1047	0	23
posx-kb	139	0	3
posx-mokobara-admin	3940	0	58
posx-mokobara-backend	1335	32	34
posx-mokobara-clearance-backend	921	0	20
posx-mokobara-store	963	0	30
posx-ugao-admin	3762	0	54
posx-ugao-backend	1568	0	35
posx-ugao-store	863	0	25

SKIPPED

REPO	REASON
posx-comet-store	no node_modules (npm install failed)
posx-demo-store	no node_modules (npm install failed)
posx-eume-store	no node_modules (npm install failed)

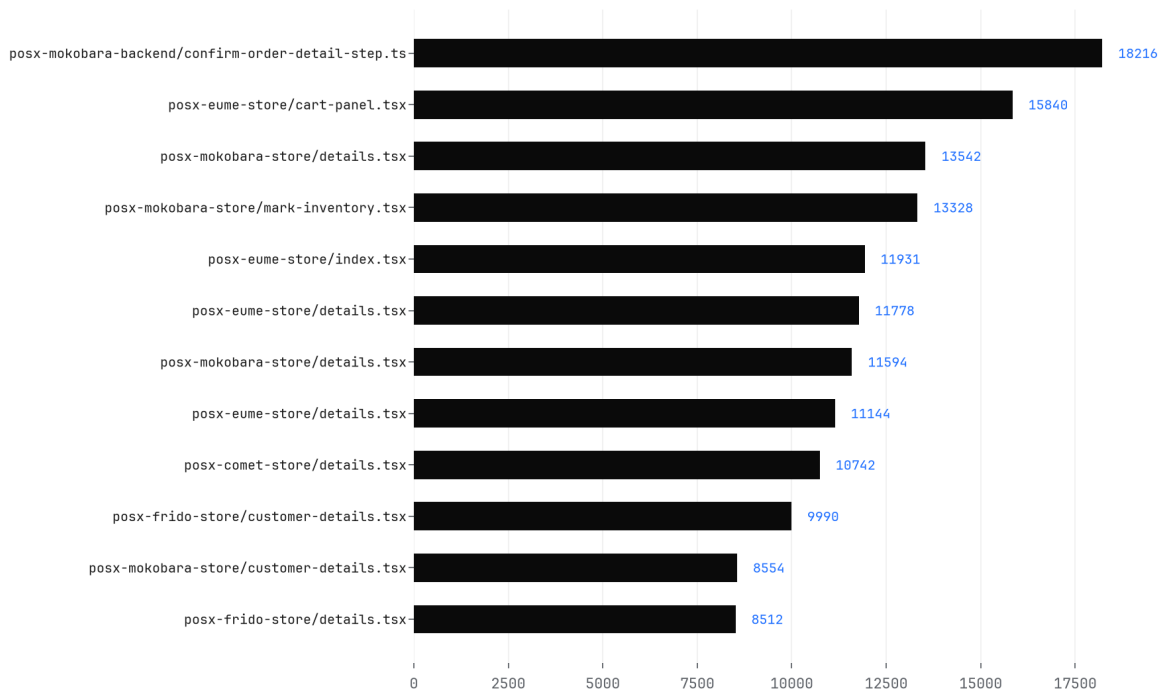
REPO	REASON
posx-frido-b2b-store	no node_modules (npm install failed)
posx-frido-mobility-store	no node_modules (npm install failed)
posx-frido-store	no node_modules (npm install failed)

2. Complexity x churn hotspots

The files where cyclomatic complexity and change frequency compound -- where defects concentrate, per Tornhill's "Your Code as a Crime Scene" methodology:

REPO	FILE	TOTAL CCN	MAX FN CCN	REVISIONS	HOTSPOT SCORE
posx-mokobara-backend	src/workflows/order-detail/steps/confirm-order-detail-step.ts	198	82	92	18216
posx-eume-store	src/components/app/product/cart-panel.tsx	330	103	48	15840
posx-mokobara-store	src/pages/exchanges/details.tsx	366	45	37	13542
posx-mokobara-store	src/pages/inventory/mark-inventory.tsx	272	19	49	13328
posx-eume-store	src/components/app/home/index.tsx	291	59	41	11931
posx-eume-store	src/pages/exchanges/details.tsx	302	21	39	11778
posx-mokobara-store	src/pages/products/details.tsx	374	61	31	11594
posx-eume-store	src/pages/products/details.tsx	398	106	28	11144
posx-comet-store	src/pages/products/details.tsx	262	107	41	10742
posx-frido-store	src/pages/online-orders/details/customer-details.tsx	370	93	27	9990
posx-mokobara-store	src/pages/orders/details/customer-details.tsx	329	67	26	8554
posx-frido-store	src/pages/products/details.tsx	266	18	32	8512

posx: top complexity x churn hotspots



3. Logical/temporal coupling

Files that change together across commits, whether or not either imports the other -- often reveals coupling an import graph alone would miss:

REPO	ENTITY	COUPLED WITH	CO-CHANGE DEGREE
posx-comet-backend	src/workflows/cart/workflows/remove-campaign-coupon.ts	src/workflows/cart/workflows/remove-promotional-coupon.ts	100
posx-comet-backend	src/workflows/cart/apply-promotional-coupon.ts	src/workflows/cart/remove-from-cart.ts	100
posx-comet-backend	src/api/order-payment/route.ts	src/api/order-product/route.ts	100
posx-comet-backend	src/workflows/cart/apply-promotional-coupon.ts	src/workflows/cart/remove-promotional-coupon.ts	100
posx-comet-backend	src/workflows/cart/remove-from-cart.ts	src/workflows/cart/remove-promotional-coupon.ts	100
posx-comet-store	src/pages/gift-card/top-up/razorpay-form.tsx	src/pages/gift-card/top-up/razorpay-pos-form.tsx	100
posx-eume-admin	src/routes/employees/employee-create/components/employee-create-form/employee-create-form.tsx	src/routes/employees/types.ts	100
posx-eume-admin	src/routes/store-coupons/coupon-create/components/coupon-create-form/coupon-create-form.tsx	src/routes/store-coupons/coupon-edit/components/edit-coupon-form.tsx	100
		src/routes/store-coupons/types.ts	100

REPO	ENTITY	COUPLED WITH	CO-CHANGE DEGREE
posx-eume-admin	src/routes/store-coupons/coupon-edit/components/edit-coupon-form.tsx		
posx-eume-admin	src/routes/store-coupons/coupon-detail/components/discount-info-section.tsx	src/routes/store-coupons/types.ts	100

4. Honest limitations

- Coupling by co-change does not distinguish "these files are properly related" from "this is an unnecessary coupling that should be refactored away" -- read the actual files before concluding either way.
- A circular-dependency count of 0 (or a low count of legitimate parent-child model pairs) does not mean the architecture has no problems -- it means this specific, narrow measurement found none. It says nothing about coupling *between* repos.

Raw data: `depgraph_summary.csv` , `complexity_hotspots.csv` , `churn_coupling.csv` , `depgraph_raw/*.json` (full per-repo graphs, regenerate via the `depgraph` module -- not archived, see *METHODOLOGY.md*).

posx — Duplication

Two independent measurements, kept separate deliberately: `jscpd` finds block-level duplication (≥ 10 lines / ≥ 70 tokens) across the whole portfolio in one pass; a direct sha256 hash of whole-file contents finds exact, byte-identical files. A file can share a `jscpd` block with another file while differing everywhere else -- conflating the two would overclaim whole-file identity from a partial match (a real mistake caught and corrected while building this tool by running an actual `diff` ; see `docs/METHODLOGY.md`). Data:

`duplication_summary.json` , `duplication_clones.csv` , `exact_duplicate_files.csv` , `exact_duplicate_summary.json` .

1. Block-level duplication (jscpd)

63.6% of all lines are part of a duplicated block. 15988 of 17658 clone pairs are cross-repo (not within a single repo).

2. Exact, byte-identical files

3001 files are byte-identical across two or more repos at the same relative path (of 26 repos scanned) -- the strongest possible signal for extracting a shared package. Max repos sharing one identical file: 9.

RELATIVE PATH (OR HASH GROUP)	REPOS SHARING IT	REPOS
<code>src/admin/vite-env.d.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/admin/tsconfig.json</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/order-detail-store-detail-link.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/football-employee-details.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/order-product-shopify-variant-link.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/football-store-details.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/shopify-collection-link.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/shopify-product-link.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/order-detail-employee-link.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
<code>src/links/order-detail-payment-detail-link.ts</code>	9	<code>posx-comet-backend</code> , <code>posx-demo-backend</code> , <code>posx-eume-backend</code> , <code>posx-frido-b2b-backend</code> , <code>posx-frido-backend</code> , <code>posx-frido-mobility-backend</code> , <code>posx-mokobara-backend</code> , <code>posx-mokobara-clearance-backend</code> , <code>posx-ugao-backend</code>
	9	

RELATIVE PATH (OR HASH GROUP)	REPOS SHARING IT	REPOS
src/links/shopify-variant-link.ts		posx-comet-backend, posx-demo-backend, posx-eume-backend, posx-frido-b2b-backend, posx-frido-backend, posx-frido-mobility-backend, posx-mokobara-backend, posx-mokobara-clearance-backend, posx-ugao-backend
src/lib/puppeteer.ts	9	posx-comet-backend, posx-demo-backend, posx-eume-backend, posx-frido-b2b-backend, posx-frido-backend, posx-frido-mobility-backend, posx-mokobara-backend, posx-mokobara-clearance-backend, posx-ugao-backend
src/lib/nodemailer.ts	9	posx-comet-backend, posx-demo-backend, posx-eume-backend, posx-frido-b2b-backend, posx-frido-backend, posx-frido-mobility-backend, posx-mokobara-backend, posx-mokobara-clearance-backend, posx-ugao-backend
src/modules/shopify-collection/index.ts	9	posx-comet-backend, posx-demo-backend, posx-eume-backend, posx-frido-b2b-backend, posx-frido-backend, posx-frido-mobility-backend, posx-mokobara-backend, posx-mokobara-clearance-backend, posx-ugao-backend
src/modules/shopify-collection/service.ts	9	posx-comet-backend, posx-demo-backend, posx-eume-backend, posx-frido-b2b-backend, posx-frido-backend, posx-frido-mobility-backend, posx-mokobara-backend, posx-mokobara-clearance-backend, posx-ugao-backend

3. Honest limitations

- jscpd's percentage is inflated by legitimate per-project boilerplate (lockfiles, generated config) that isn't a real duplication problem -- the exact-match list above is the stricter, more actionable signal.
- Exact-match groups only catch identical files at the *same relative path*. Two files with identical content at different paths (e.g. one repo renamed its copy) are not detected here.

Raw data: `duplication_clones.csv` , `exact_duplicate_files.csv` .

posx — Security

`gitleaks` over complete git history per repo (a secret removed in a later commit is still permanently readable by anyone with clone access) + `semgrep` (`p/security-audit` + `p/secrets`) over the current tree. Data: `security_secrets.csv` , `security_semgrep.csv` , `security_summary.json` .

1. Secret-history scan (gitleaks)

80 findings across 8 repos (26 repos scanned):

RULE	COUNT
generic-api-key	37
curl-auth-header	20
shopify-access-token	17
aws-access-token	3
shopify-custom-access-token	2
jwt	1

By repo:

REPO	FINDINGS
posx-mokobara-backend	29
posx-comet-backend	24
posx-ugao-backend	11
posx-kb	10
posx-frido-store	2
posx-mokobara-clearance-backend	2
posx-eume-backend	1
posx-mokobara-store	1

Every rule-format match above is a starting point, not a confirmed secret -- generic-entropy rules in particular need a manual look. This tool does not attempt to validate or use any discovered credential against a live provider API (out of scope regardless of feasibility); it also checks whether each is still present in current `HEAD` vs. history-only where that distinction changes the remediation urgency -- see the raw CSV's `commit` column and check with `git grep <secret-prefix> HEAD` in the affected repo.

2. Code-pattern scan (semgrep)

2 findings.

SEVERITY	COUNT
WARNING	2

3. Honest limitations

- gitleaks' `generic-api-key` and `curl-auth-header` rules are pattern/entropy-based and will include false positives (test fixtures, non-secret high-entropy strings). Only format-confirmed rules (`aws-access-token` , `shopify-access-token` , `jwt` , etc.) are close to self-verifying by their prefix alone.
- semgrep's ruleset here (`p/security-audit` , `p/secrets`) is broad but not exhaustive -- absence of a finding is not proof of absence of a vulnerability class the ruleset doesn't cover.

Raw data: `security_secrets.csv` , `security_semgrep.csv` .

posx — Dependency *Health*

`osv-scanner` (Google's cross-ecosystem OSV-database scanner) reads each repo's lockfile directly against known CVE records -- a different, deeper question than gitleaks (committed secrets) or semgrep (code patterns): are the dependencies themselves known-vulnerable. `npm outdated` gives current vs. latest per package. `npm audit` was tried and dropped (hung repeatedly on a live registry round-trip in this environment); osv-scanner is faster and already confirmed working -- see docs/METHODOLOGY.md. Data: `deps_cves.csv` , `deps_outdated.csv` .

1. Known-CVE findings

2636 findings across 18 repos.

SEVERITY	COUNT
critical	63
high	1241
medium	1107
low	225

REPO	CVE FINDINGS
posx-demo-backend	543
posx-comet-backend	308
posx-ugao0-backend	295
posx-frido-b2b-backend	192
posx-frido-mobility-backend	192
posx-frido-backend	161
posx-ugao0-admin	139
posx-mokobara-store	102
posx-ugao0-store	98
posx-frido-admin	94
posx-frido-b2b-admin	94
posx-mokobara-admin	94

2. Package staleness

772 outdated packages, 419 a full major version behind (current vs. latest):

REPO	PACKAGE	CURRENT	LATEST
posx-comet-admin	@dnd-kit/sortable	8.0.0	10.0.0
posx-comet-admin	@hookform/resolvers	3.4.2	5.9.1
posx-comet-admin	@tanstack/react-table	8.20.5	9.2.4
posx-comet-admin	@types/node	20.17.24	26.4.1

REPO	PACKAGE	CURRENT	LATEST
posx-comet-admin	@types/react	18.3.18	19.2.18
posx-comet-admin	@types/react-dom	18.3.5	19.2.7
posx-comet-admin	@vitejs/plugin-react	4.2.1	6.1.1
posx-comet-admin	cmdk	0.2.1	1.1.1
posx-comet-admin	date-fns	3.6.0	4.4.0
posx-comet-admin	i18next	23.7.11	26.4.2
posx-comet-admin	i18next-browser-languagedetector	7.2.0	8.2.1
posx-comet-admin	i18next-http-backend	2.4.2	4.0.2

3. Honest limitations

- osv-scanner flags a *known* CVE affecting the resolved version -- it does not confirm the vulnerable code path is actually reachable/ exploited in this codebase's usage of the package.
- "Major version behind" is a proxy for staleness/upgrade risk, not itself a defect -- some majors are trivial bumps, others are breaking rewrites. Triage by package, not by count.

Raw data: `deps_cves.csv` , `deps_outdated.csv` .

posx — Code *Quality* (Static Analysis)

Each repo's own linter, run with its own config -- never a config this tool invents. ESLint via the repo's local `node_modules/.bin/eslint` (not `npx`, which cannot resolve a project's plugin-dependent config). `ruff` for Python, `staticcheck` for Go. Data: `lint_quality.csv`.

1. Results

7 of 26 repos actually linted (19 skipped -- see below). **4165 errors, 218 warnings** total.

REPO	LINTER	ERRORS	WARNINGS	TOP RULES
posx-mokobara-backend	eslint	2634	127	@typescript-eslint/no-explicit-any:1824;@typescript-eslint/ban-ts-comment:247;@t
posx-frido-backend	eslint	568	13	@typescript-eslint/no-explicit-any:368;@typescript-eslint/ban-ts-comment:102;@ty
posx-frido-mobility-backend	eslint	389	0	@typescript-eslint/no-explicit-any:211;@typescript-eslint/ban-ts-comment:91;@typ
posx-frido-b2b-backend	eslint	259	0	@typescript-eslint/no-explicit-any:145;@typescript-eslint/ban-ts-comment:58;@typ
posx-mokobara-store	eslint	192	39	@typescript-eslint/no-explicit-any:183;react-hooks/exhaustive-deps:31;@typescrip
posx-ugaoo-store	eslint	119	37	@typescript-eslint/no-explicit-any:106;react-hooks/exhaustive-deps:30;@typescrip
posx-kb	eslint	4	2	react-hooks/set-state-in-effect:3;@typescript-eslint/no-unused-vars:2;react-hook

2. Why repos were skipped

19 repos could not be linted -- almost always because the repo's own `package.json` declares a `lint` script that calls a linter never installed as a real dependency (`eslint` not in `node_modules/.bin`). That is itself a finding: a lint script that cannot run is not a quality gate, it is a broken promise in `package.json`.

3. Honest limitations

- Different repos may run different rule strictness (each uses its own config) -- error counts are not directly comparable across repos with different configs.

Raw data: `lint_quality.csv`.

posx — Test *Quality*

Not "has a test script" (a proxy) -- every repo's unit-test script was actually executed and its real pass/fail summary parsed from the runner's own output. Scoped to unit tests only (integration suites needing live infrastructure are out of scope for an unattended run -- see [docs/METHODOLOGY.md](#)). CI-gate status is from parsing actual workflow step contents, not inferred from a workflow file's existence. Data: `testquality_runs.csv`, `ci_gates.csv`.

1. CI enforcement

1 of 26 repos have a CI workflow that runs tests on any trigger. The rest configure CI for build/deploy only.

2. Real, executed test results

REPO	SCRIPT	RUNNER	PASSED	FAILED	TOTAL	NODE USED	NOTE
posx-comet-admin	test	vitest	1	0	1	v20.20.2	
posx-comet-backend	test:unit	jest	3	3	6	v20.20.2	
posx-demo-admin	test	vitest	0	1	1	v20.20.2	
posx-demo-backend	test:unit	jest	0	0	0	v20.20.2	jest ran, testMatch pattern matched zero files --
posx-eume-admin	test	vitest	0	1	1	v20.20.2	
posx-eume-backend	test:unit	jest	169	10	179	v20.20.2	
posx-frido-admin	test	vitest	17	1	18	v20.20.2	
posx-frido-b2b-admin	test	vitest	1	0	1	v20.20.2	
posx-frido-b2b-backend	test:unit	jest	0	0	0	v20.20.2	jest config validation error -- suite never starte
posx-frido-backend	test:unit	jest	36	0	36	v20.20.2	
posx-frido-mobility-admin	test	vitest	0	1	1	v20.20.2	
posx-frido-mobility-backend	test:unit	jest	0	0	0	v20.20.2	jest ran, testMatch pattern matched zero files --
posx-mokobara-admin	test	vitest	0	1	1	v20.20.2	
posx-mokobara-backend	test:unit	jest	162	0	162	v20.20.2	
posx-mokobara-clearance-backend	test:unit	jest	0	0	0	v20.20.2	jest ran, testMatch pattern matched zero files --
posx-ugaoo-admin	test	vitest	64	6	70	v20.20.2	
posx-ugaoo-backend	test:unit	jest	852	7	859	v20.20.2	

4 repos fully passing right now. **9** have real failures right now. **3** have a working test harness that matches zero actual test files. **1** have a broken test configuration that prevents the suite from starting at all. **9** have no unit-test script.

3. Honest limitations

- Integration/e2e test scripts are not executed by this module (they typically need live infrastructure this analysis has no access to) -- a repo could have a much larger, currently broken integration suite invisible here.

- A repo's runtime environment (Node/Python/Go version) can cause a false failure unrelated to the code under test -- this tool pins to the target's own declared engine version where it can detect one and records which runtime actually ran each suite (`node_version_used` / equivalent), but a mismatch it can't auto-correct will still show as a false failure. Check the raw log before treating any single failure as confirmed.

Raw data: `testquality_runs.csv` , `ci_gates.csv` . Raw logs regenerate via the `testquality` module (not archived).

posx — *Mutation* Testing

Stryker: injects synthetic bugs (mutants) into a file and re-runs the test suite against each one. A test suite that passes on unmutated code but also passes on a mutated one has not actually verified that behavior -- mutation score (killed / covered mutants) answers "do these tests catch a real defect," which passing-test-count alone cannot. Scoped deliberately: only repos with a fully-passing unit suite were attempted (a failing suite has no meaningful baseline), mutating only each repo's #1 complexity x churn hotspot (bounded runtime -- see docs/METHODOLOGY.md for the install recipe this took three prior failed attempts to find). Data:

`mutation_results.csv` .

1. Results

REPO	FILE MUTATED	MUTANTS	KILLED	SURVIVED	NO COVERAGE	SCORE %
posx-mokobara-backend	src/workflows/order-detail/steps/confirm-order-detail-step.ts	5	2	1	2	66.7

Scoped to the 4 repos with a fully-passing unit suite, mutating each repo's #1 complexity x churn hotspot only (bounded runtime -- Stryker needs a fresh local install per repo). 1 of 4 succeeded; the other 3 failed for repo-specific reasons (vitest-runner config, frido-backend jest setup) not chased further given time -- see `mutation_results.csv` `skip_reason` per row.

2. Honest limitations

- Attempted on 4 repos, succeeded on 1 -- the others failed for repo-specific environment reasons (see `mutation_summary.json` 's `skip_reasons`), not chased further given time. This is a real, if narrow, data point per succeeding repo, not a portfolio-wide mutation posture.
- Only one file per repo was mutated (the highest hotspot) -- a low mutation score there is a real, specific finding about that file; it should not be generalized to "this repo's tests are bad" without checking other files.

Raw data: `mutation_results.csv` .

posx — Consolidation & *Way-Forward* Roadmap

Synthesizes every other category into one ranked list and a prioritized action list. The risk score (`synthesize.py`) is a weighted, min-max-normalized combination of escape rate, hotspot concentration, duplication involvement, missing CI gates, test failures, security findings, and bus-factor -- a heuristic for prioritization, stated as such, not a ground truth. Data: `risk_ranking.csv` , `risk_weights.json` .

1. Highest-priority repos, by composite risk score

REPO	RISK SCORE	ESCAPES	TOP-5 HOTSPOT SUM	EXACT DUP FILES	CI GATE MISSING	SECURITY FINDINGS
posx-comet-backend	0.5782	801	16233	245	True	24
posx-mokobara-store	0.5569	740	54442	107	True	1
posx-mokobara-backend	0.5307	979	40200	252	False	29
posx-mokobara-admin	0.5218	267	11542	1634	True	1
posx-comet-store	0.512	917	29891	123	True	0
posx-eume-store	0.4394	235	57587	224	True	0
posx-frido-backend	0.4362	666	17015	486	True	0
posx-eume-admin	0.4278	41	4375	1653	True	0
posx-demo-admin	0.42	1	835	1669	True	0
posx-frido-mobility-admin	0.4188	0	1133	1731	True	0

See `risk_weights.json` for the exact weights behind this ranking -- change them if you weigh these dimensions differently; the raw per-dimension numbers underneath are the actual evidence.

2. Recommended action order

- 1. Rotate credentials found in git history first.** 80 secret-shaped findings across 8 repos -- see `SECURITY.md` . This is a today action, independent of everything else here.
- 2. Wire CI to actually run tests on the 24 repos where it currently doesn't.** A working, proven pattern already exists in this portfolio: `posx-mokobara-backend`. This is the single highest-leverage fix available: every other quality finding downstream of 'nothing blocks a bad merge' gets caught here, going forward, for free.
- 3. Extract the byte-identical shared code into real internal packages.** 3001 files are identical across multiple repos (up to 9 at once) -- see `DUPLICATION.md` for the exact file list. Extracting only what the exact-match list confirms identical changes zero behavior; verify with `diff` before assuming a jscpd block match means the same for any file not on the exact-match list.
- 3 repos have a working test harness and zero tests behind it.** Cheaper to fix than repos with no harness at all -- the wiring already works, someone just needs to write the tests.
- 5. 72 candidate toil clusters** (3060 commits) are repeated, mechanical work concentrated by location -- see `EFFORT_ALLOCATION.md` for the ranked list. Each is worth a human read before committing to a specific fix; this tool does not size the remediation effort.

3. Honest limitations

- The composite risk score is dominated in part by raw activity/scale (an old, high-churn repo accumulates more escapes and hotspot mass than a small quiet one just by having existed longer and shipped more) -- it is a prioritization aid for where to look first, not a cleanly isolated "badness" measure. Read the underlying per-dimension columns before acting on rank alone.
- This report does not estimate effort or ROI for any recommendation -- that requires product and team context this tool does not have. It tells you what the data shows and where to look; the sizing and sequencing judgment is a human decision.

Raw data: `risk_ranking.csv` , `risk_weights.json` .

posx — Method *Review*

This tool's own self-audit: which modules ran, which failed, and what has been independently checked rather than taken on faith. See docs/METHODOLOGY.md for the full list of bugs found and fixed while building this tool -- kept visible rather than quietly corrected away.

Module run status

MODULE	STATUS	ELAPSED (S)
knowledge_graph	ok	0.3

All 1 modules completed.

What has NOT been independently verified in this run

- Composite risk ranking weights (`synthesize.py`) are a stated judgment call, not empirically validated against any outcome.
- Toil clusters (`effort.py`) are mechanically grouped, not diff-read and confirmed.
- Security findings below a format-confirmed rule (generic-api-key, curl-auth-header) are not individually triaged for true/false-positive status.

This section exists so a reader can tell which numbers in this report carry independent verification and which are this tool's first-pass output -- treat the latter as a lead to check, not a settled fact.